Міністерство освіти та науки України

Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №1
з дисципліни “Програмування мовою Dart”

на тему: “Функції та обробка колекцій”

Виконала:

ст. гр. ПЗПІ-23-3 Василенко С. О.

Перевірив:

асистент каф ПІ Баличев М. Д.

Харків 2026

1 ФУНКЦІЇ ТА ОБРОБКА КОЛЕКЦІЙ

1.1 Мета роботи

Опанувати оголошення функцій із позиційними та іменованими параметрами, обробку колекцій засобами Iterable, використання рядків, регулярних виразів, замикань і дат у консольному застосунку мовою Dart.

1.2 Хід роботи

1.2.1 Постановка завдання

Відповідно до індивідуального варіанта предметною областю обрано розклад занять групи. Кожен запис містить назву дисципліни, викладача, тип заняття, день тижня, час початку та аудиторію. Для виконання вимоги про арифметику дат додатково включено дату проведення заняття.

Потрібно створити програму, яка зчитує константний набір із щонайменше 15 рядків, знаходить пошкоджені записи, перетворює придатні значення на типізовані записи та перевіряє правила предметної області. У реалізації першої лабораторної власні класи не використовуються.

Далі потрібно виконати фільтрацію та агрегування колекцій, групування за категорією, сортування за двома критеріями, формування текстового звіту, продемонструвати замикання, регулярний вираз з іменованими групами, заміну за збігом і визначення різниці між датами.

Результат має успішно запускатися, бути відформатованим командою

dart format і не містити зауважень статичного аналізатора dart analyze.

Вихідний код та історію розробки розміщено в GitHub-репозиторії:

https://github.com/NureVasylenkoSofiia/dart-vasylenko-sofiia

1.2.2 Структура програми та вибір типів даних

Проєкт створено стандартною командою dart create -t console group_schedule. Точка входу розташована у bin/group_schedule.dart, де організовано послідовність обробки даних і консольне виведення. Окремі
функції рознесено між файлами папки lib/, щоб спростити читання програми та подальший перехід до об’єктно-орієнтованої реалізації в ЛР2

Таблиця 1.1 – Розподіл функцій між файлами

Файл	Призначення
schedule_data.dart	Початковий константний перелік рядків
schedule_parser.dart	Опис Record і розбір одного рядка
schedule_validator.dart	Змістова перевірка заняття
schedule_statistics.dart	Фільтрація, статистика, групування, сортування
schedule_report.dart	Побудова текстового звіту
schedule_utils.dart	Замикання, RegExp, маскування й дати

Початкові дані описані як const List<String>. Для структурованого

заняття використано Record із псевдонімом типу ScheduleEntry. Такий вибір відповідає обмеженню методички: класів у ЛР1 не потрібно, але названі поля й статичні типи роблять доступ до значень зрозумілим.

Лістинг 1.1 – Типізований запис заняття:

```
typedef ScheduleEntry = ({
String subject,
String teacher,
String lessonType,
String weekday,
int startMinutes,
String room,
DateTime date,
});
```

Час подано як кількість хвилин від початку доби у int; дату – як DateTime. Для розкладу за днями використано Map<String, List<ScheduleEntry>>, для унікальних дисциплін – множину Set.

1.2.3 Розбір та перевірка вхідних даних

Вхідні рядки складаються із семи полів, розділених символом «;». Із 15 підготовлених записів 12 є коректними, а три навмисно містять різні типи пошкоджень: відсутність поля аудиторії, текст замість часу і неможливу дату. Це дозволяє перевірити стійкість програми до некоректного формату.

Функція parseScheduleLine() спочатку застосовує split і перевіряє кількість частин. Для числових складових часу використано int.tryParse, для дати – DateTime.tryParse. Функція повертає ScheduleEntry?: у разі помилки – null, без винятку.

Лістинг 1.2 – Перевірка роздільників і перетворення значень:

```
final fields = line.split(';');
if (fields.length != 7) {
return null;
}
final hours = int.tryParse(timeParts[0]);
final minutes = int.tryParse(timeParts[1]);
final date = DateTime.tryParse(dateText);
if (hours == null || minutes == null || date == null) {
return null;
}
```

Окремо перевіряється нормалізований текст дати у вигляді YYYY-MM-DD. Це потрібно, тому що парсер дат може нормалізувати некоректний день місяця. Далі відсіюються години поза діапазоном 0–23 та хвилини поза діапазоном 0-59.

Після успішного розбору функція validateScheduleEntry() перевіряє
дозволені типи занять, назву дня, його відповідність даті, час проведення в межах 08:00-20:00 та формат аудиторії. На відміну від парсера, вона повертає List<String> усіх знайдених порушень, а не зупиняється після першого.

Лістинг 1.3 – Приклад змістової валідації:

```
if (!weekdays.contains(entry.weekday)) {
errors.add('Некоректний день тижня');
} else if (weekdays[entry.date.weekday - 1] !=
entry.weekday) {
errors.add('День тижня не відповідає даті');
}
```

1.2.4 Обробка колекцій та агрегування

Для обчислення показників використано стандартні операції Iterable. Метод where() відбирає лабораторні заняття; map() переводить записи в назви дисциплін; fold() підсумовує кількість занять. Це відповідає вимозі використати конвеєри обробки замість циклу з ручним накопиченням.

Лістинг 1.4 – Конвеєр where → map → fold:

```
int countLessons(List<ScheduleEntry> entries) {
return entries
.where((entry) => entry.startMinutes >= 0)
.map((entry) => 1)
.fold<int>(0, (total, value) => total + value);
}
```

Під час виконання отримано чотири лабораторні заняття та шість унікальних дисциплін із дванадцяти прийнятих записів. Унікальність назв досягається переходом від результату map() до множини через toSet().

Для групування застосовано Map<String, List<ScheduleEntry>>.

Ключ – день тижня, а значення – перелік занять. Виклик putIfAbsent() створює список, якщо для дня ще немає запису. Кількість занять у кожній групі згортається методом fold().

Лістинг 1.5 – Конвеєр where → map → fold:

```
for (final entry in entries) {
groups.putIfAbsent(entry.weekday, () =>
<ScheduleEntry>[]);
groups[entry.weekday]?.add(entry);
}
```

Таблиця 1.2 – Розподіл коректних занять

День	Занять
Понеділок	3
Вівторок	3
Середа	3

Четвер	2
П’ятниця	1

1.2.5 Сортування та формування звіту

У програмі реалізовано два окремі способи сортування: за часом початку та за назвою дисципліни. Обидві функції мають іменований параметр ascending зі значенням true. Під час сортування створюється копія списку, тому початкові дані не змінюють свій порядок.

Лістинг 1.6 – Сортування за часом із вибором напряму

```
final result = List<ScheduleEntry>.of(entries);
result.sort((first, second) {
final comparison =
first.startMinutes.compareTo(second.startMinutes);
return ascending ? comparison : -comparison;
});
return result;
```

У звіті виводяться п’ять перших занять після сортування за часом; для іншої демонстрації застосовано сортування дисциплін за спаданням. За однакового значення ключа порядок записів додатково не визначається. Функція buildScheduleReport() повертає підготовлений текст і використовує три іменовані параметри: topCount = 5, ascending = true та separator = " | ". Текст формується через StringBuffer, щоб уникнути багаторазового створення проміжних рядків операцією конкатенації.

Лістинг 1.7 – Початок формування текстового звіту:

```
final buffer = StringBuffer();
final sorted = sortByStartTime(entries, ascending:
ascending);
final selected = sorted.take(topCount);
buffer.writeln('=== ЗВІТ ПРО РОЗКЛАД ЗАНЯТЬ ===');
buffer.writeln('Загальна кількість занять:
${entries.length}');
buffer.writeln('Показано записів: ${selected.length}');
```

Функція повертає сформований результат через buffer.toString(), а його виведення виконується в точці входу. Отже, логіку форматування можна застосувати повторно без залежності від консольного інтерфейсу.

1.2.5 Замикання, регулярні вирази та дати

Для нумерації вхідних рядків створено функцію createRecordCounter(), яка повертає іншу функцію. Внутрішня функція зберігає доступ до локальної змінної count після повернення із зовнішньої функції. Таким чином реалізовано замикання; кожний його виклик збільшує порядковий номер.

Лістинг 1.8 – Замикання для нумерації записів:

```
int Function() createRecordCounter() {
var count = 0;
return () {
count++;
return count;
};
}
```

Номери аудиторій перевіряються регулярним виразом, записаним як сирий рядок із групами building та number. Наприклад, рядок «A-101» проходить перевірку, а «A101» – ні. Метод

replaceAllMapped() приховує перші дві цифри, залишаючи корпус і останню цифру.

Лістинг 1.9 – Регулярний вираз та отримання іменованої групи:

```
final RegExp roomPattern = RegExp(
r'^(?<building>[A-ЯІЇЄҐA-Z])-(?<number>\d{3})$',
);
final regexMatch = match as RegExpMatch;
final number = regexMatch.namedGroup('number') ?? '';
```

Для визначення віку заняття функція daysSinceLesson() перетворює обидві дати на північ UTC і використовує DateTime.difference(). Результат Duration.inDays показує різницю календарних днів. Необов’язковий параметр currentDate дозволяє передати фіксовану дату під час перевірки.

Лістинг 1.10 – Різниця між датами в днях:

```
final today = DateTime.utc(now.year, now.month, now.day);
final Duration difference = today.difference(lessonDay);
return difference.inDays;
```

1.2.6 Результати виконання програми

Програму запущено з кореня проєкту командою dart run. У консольному виводі підтверджено перевірку всіх вхідних рядків: 12 записів успішно розібрано та перевірено, три записи з пошкодженим форматом відхилено без аварійного завершення.

Лістинг 1.11 – Підсумок консольного виконання:

```
=== ПІДСУМКИ ===
Коректних записів: 12
Відхилених записів: 3
```

```
=== СТАТИСТИКА РОЗКЛАДУ ===
Лабораторних занять: 4
Унікальних дисциплін: 6
Загальна кількість занять: 12
```

Правильність маскування продемонстровано на п’яти аудиторіях. На скриншоті з реального запуску показані також результати обчислення віку занять, одержані 7 листопада 2026 року.

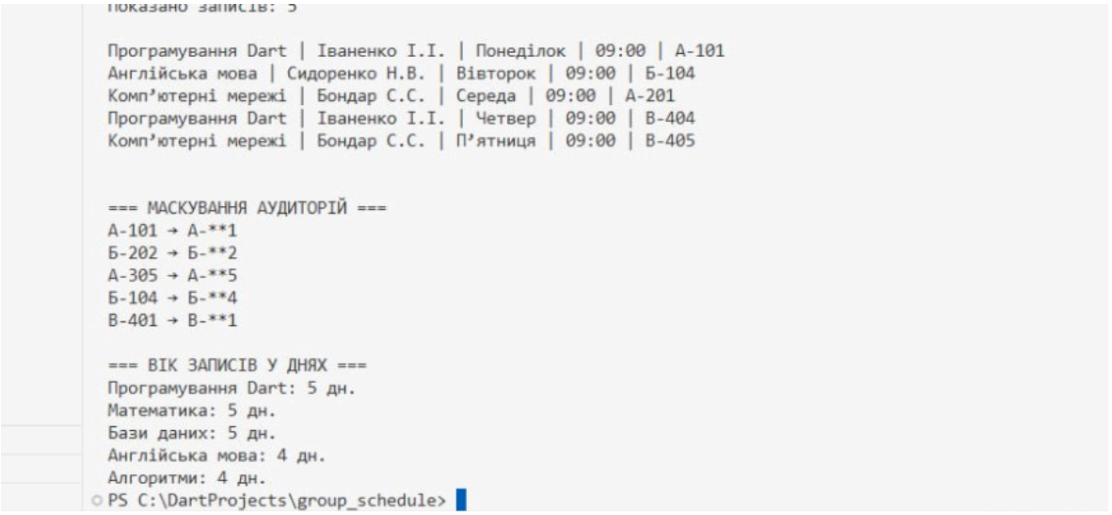


Рисунок 1.1 – Результати маскування та арифметики дат у VS Code

1.2.7 Перевірка якості коду

Для контролю оформлення виконано команду dart format ., що перевірила дев’ять файлів і не виявила необхідності у виправленнях форматування. Команда dart analyze завершилася без попереджень і помилок. Її результат збережено у файлі analyze_result.txt у корені репозиторію.

Лістинг 1.12 – Збережений результат статичного аналізу:

```
Analyzing group_schedule...
No issues found!
```

У analysis_options.yaml підключено рекомендований набір правил package:lints/recommended.yaml. Публічні функції та типи забезпечено документаційними коментарями. Під час розбору вхідних значень використано nullable-тип, перевірки на null та tryParse.

У прикладній логіці не застосовано late, dynamic або оператор примусового розпакування nullable-значень. Вираз as RegExpMatch використано в маскуванні для доступу до namedGroup() результату регулярного виразу. Повний код, конфігураційні файли, README та Git-історія доступні у відкритому
репозиторії.

1.2.8 Письмове обґрунтування обраного підходу

Основне рішення полягало у відокремленні сирих рядків, їх розбору, перевірки та подальших перетворень. Це дозволило не змішувати роботу з текстом із розрахунками статистики. Відхилені рядки не потрапляють у список коректних записів, тому всі підсумкові операції виконуються над уже перевіреними даними.

Для подання занять обрано Record, оскільки він зберігає типізацію й дозволяє використовувати назви полів без власного класу. При цьому записи не містять поведінки; у наступній лабораторній їх доцільно замінити класами, коли потрібно буде поєднати дані, перевірки та операції над ними.

Для часу використано кількість хвилин від початку доби, а не рядок HH:mm: так порівняння не залежить від текстового представлення часу. Дати залишено як DateTime, оскільки методичка вимагає обчислювати різницю через Duration. Для групування Map

зберігає безпосередній зв’язок між днем і заняттями; Set допомагає прибрати повтори дисциплін.

Операції where, map і fold зробили розрахунки компактними та зрозумілими. Звичайні цикли залишено в парсингу та виведенні повідомлень, де необхідно послідовно повідомляти про конкретний рядок і можливу помилку. У функції звіту використано StringBuffer, оскільки він призначений для поступового додавання фрагментів тексту.

Сторонній CSV-пакет у цій роботі не потрібний, тому що дані мають фіксований нескладний формат і роздільник «;». Із шаблонних залежностей lints потрібен для забезпечення стилю коду, а test залишено для розширення автоматизованих перевірок у майбутньому. Такий підхід дозволяє виконати вимоги першої лабораторної без зайвого ускладнення структури.

Висновки

У ході лабораторної роботи було розроблено консольну програму мовою Dart для обробки розкладу занять групи. Для представлення структурованих даних використано типізовані записи Record замість класів, що відповідає умовам першої лабораторної. Початковий набір містив 15 рядків, із яких 12 було прийнято, а три пошкоджені записи коректно відхилено.

Реалізовано розбір через split та tryParse, перевірку предметних обмежень, фільтрацію й агрегацію колекцій, групування за днями тижня та сортування за двома критеріями. Також застосовано функцію з іменованими параметрами, StringBuffer, замикання для лічильника, регулярний вираз з іменованими групами і арифметику дат через DateTime.difference та Duration.

Під час перевірки встановлено, що програма правильно обчислює статистику: чотири лабораторні заняття, шість унікальних дисциплін і дванадцять прийнятих занять. Форматування виконано командою dart format, а статичний аналізатор завершив роботу повідомленням No issues found!. Отриману реалізацію та історію змін розміщено у GitHub для подальшого розвитку в наступних лабораторних роботах.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Методичні вказівки до лабораторних робіт із дисципліни «Програмування мовою Dart», розділи 1, 2 (матеріали, надані викладачем).

2. Dart documentation. Language overview.
URL: https://dart.dev/language

3. Dart documentation. Records. URL: https://dart.dev/language/records

4. Dart API. Iterable class.
URL: https://api.dart.dev/dart-core/Iterable-class.html

5. Dart API. RegExp class.
URL: https://api.dart.dev/dart-core/RegExp-class.html

6. Dart API. DateTime class.
URL: https://api.dart.dev/dart-core/DateTime-class.html

7. Effective Dart. Style and Usage. URL: https://dart.dev/effective-dart

8. GitHub. Репозиторій «dart-vasylenko-sofiia». URL:
https://github.com/NureVasylenkoSofiia/dart-vasylenko-sofiia